

Transformers and LLMs

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

Email: oybekeraliev7@gmail.com



Learning Objectives

By the end of this lecture, students will be able to:

- Understand the motivation behind Transformers
- Explain attention mechanisms
- Know the encoder–decoder architecture
- Describe how GPT-like and BERT-like models work
- Understand LLM training

Why Transformers?

Limitations of previous architectures:

- **RNN/LSTM**: slow, sequential, long-term memory weakness
- **CNN**: good at local patterns, weak at long-range dependencies
- **Attention**: enables direct relationships between any tokens
→ Transformers enable **parallelization and global context modeling**

Key Idea: Attention

“Pay more attention to important words.”

Given query Q , keys K , values V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Benefits:

- Captures long-range dependencies
- Flexible
- Highly parallel

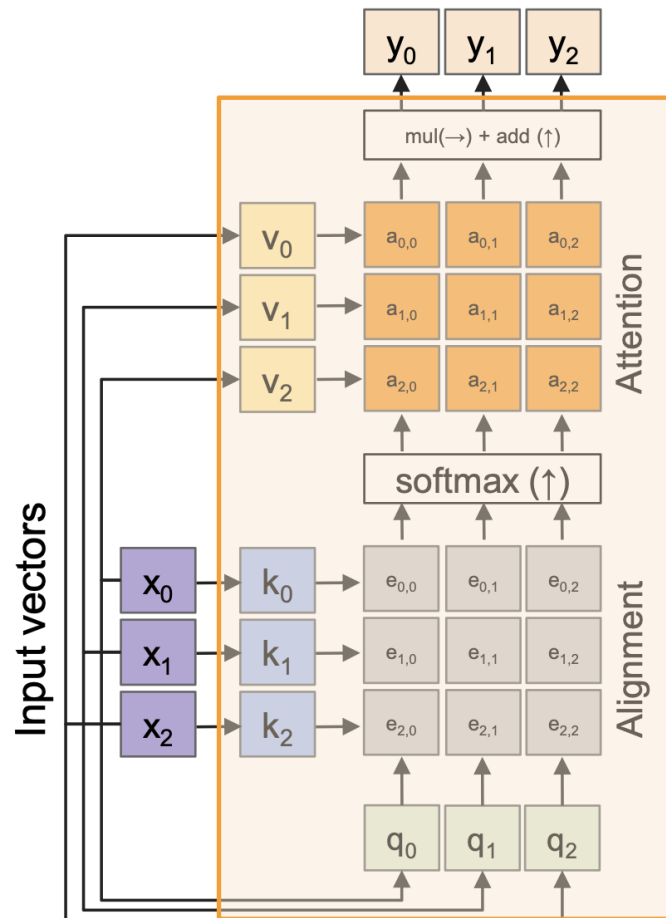
Key Idea: Attention

- **Self-attention in transformers** is a technique designed to enhance input representations by enabling each position in a sequence to engage with and determine the relevance of every other position within the same sequence

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Transformers

Attention Mechanism



Outputs:

context vectors: $\mathbf{y}$ (shape: D_v)

Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$

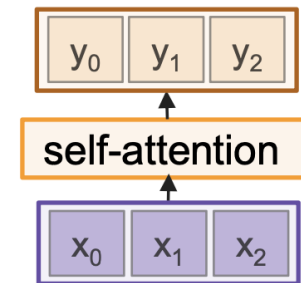
Alignment: $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)



Multi-Head Attention

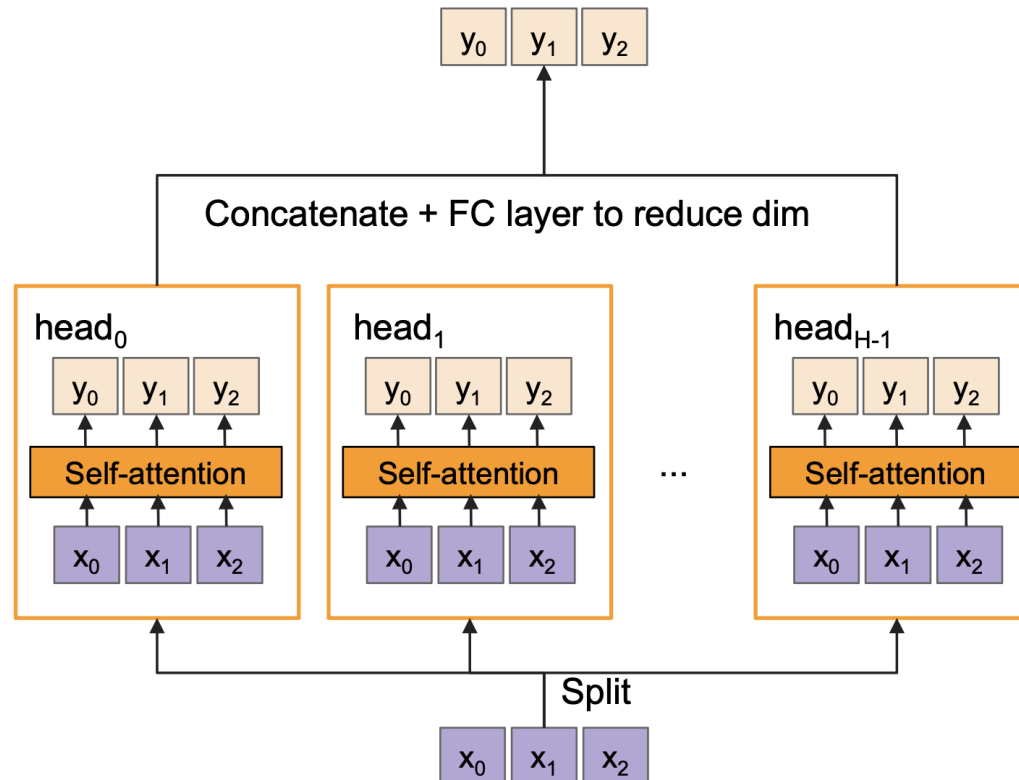
Why multi-heads?

- Each head learns different relationships
- Example:
 - Head 1 → subject–verb
 - Head 2 → coreference
 - Head 3 → positional relations

Multi-Head Attention

Multi-head self-attention layer

- Multiple self-attention “heads” in parallel



A: We may want to have multiple sets of queries/keys/values calculated in the layer. This is a similar idea to having multiple conv filters learned in a layer

Transformer Architecture

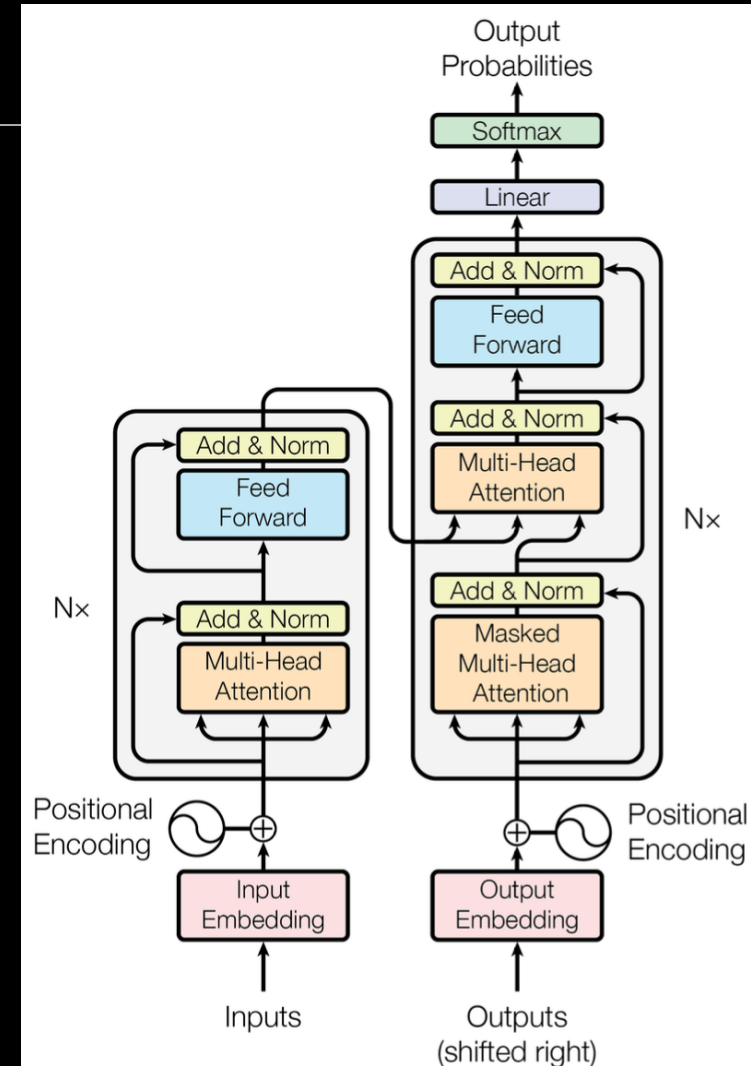
Two major components:

1. Encoder

- Self-attention
- Feed Forward
- LayerNorm + Residual

2. Decoder

- **Masked** self-attention
- Encoder–decoder attention
- Feed Forward

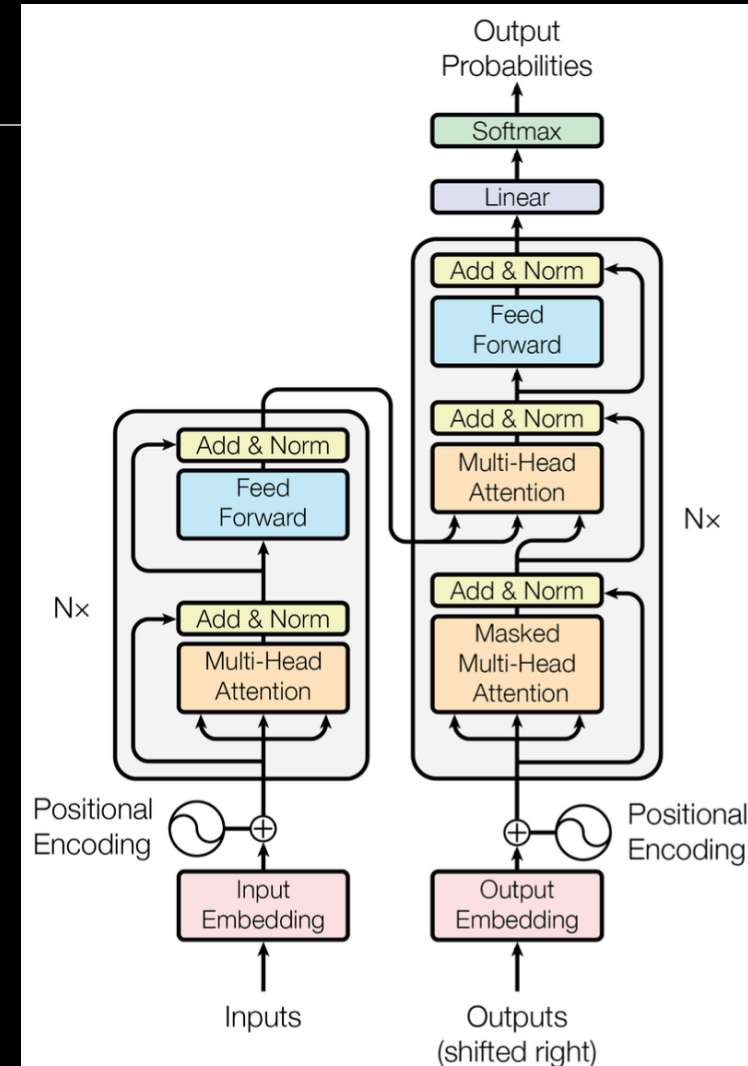


What Does Masked Attention Do?

Masked attention prevents a token from seeing future tokens.

Used in **autoregressive generation**:

- Language modeling
- GPT models
- Text generation
- Code generation



Where Encoders and Decoders are Used

Encoders (BERT-type models):

- Classification
- Sentiment analysis
- Named entity recognition
- Embedding creation

Encoder–Decoder (T5, BART):

- Translation
- Summarization
- Paraphrasing

Decoders (GPT-type models):

- Text generation
- Chatbots
- Story writing
- Dialog systems

Types of Modern LLMs

Decoder-only

- GPT-3, GPT-4, LLaMA, Falcon
- Best for generation

Encoder-only

- BERT, RoBERTa
- Best for classification / embeddings

Encoder–decoder

- T5, BART
- Best for summarization/translation/compression

Scaling Laws

LLM performance improves with:

- More parameters
- More data
- More compute
- Better training recipes

"Scaling makes models smarter."

Tokenization

Tokenization methods:

- BPE (Byte Pair Encoding)
- WordPiece
- SentencePiece
- TikToken for GPT

Tokens $\neq$ words (e.g., “Smartphone” $\rightarrow$ “Smart”, “phone”)

Tokenization

Byte-Pair Encoding (BPE)

Byte-Pair Encoding (BPE) is a text tokenization technique in Natural Language Processing. It breaks down words into smaller, meaningful pieces called subwords. It works by repeatedly finding the most common pairs of characters in the text and combining them into a new subword until the vocabulary reaches a desired size.

This technique helps in handling rare or unknown words by breaking them into smaller parts that the model has already learned during training. By reducing the vocabulary size, it makes it easier to work with large amounts of text while allowing the model to understand wide variety of languages.

Tokenization

Concepts related to BPE

To understand BPE better, it's important to know its key concepts:

- 1. Vocabulary:** In BPE vocabulary refers to the set of subword units (tokens) used to represent all the words in the corpus. After applying BPE, vocabulary consists of all the subwords that can be used to represent a word in the dataset.
- 2. Byte:** It is a basic unit of digital information that consists of 8 bits. It is used to represent characters that are encoded as bytes that will be merged.

Tokenization

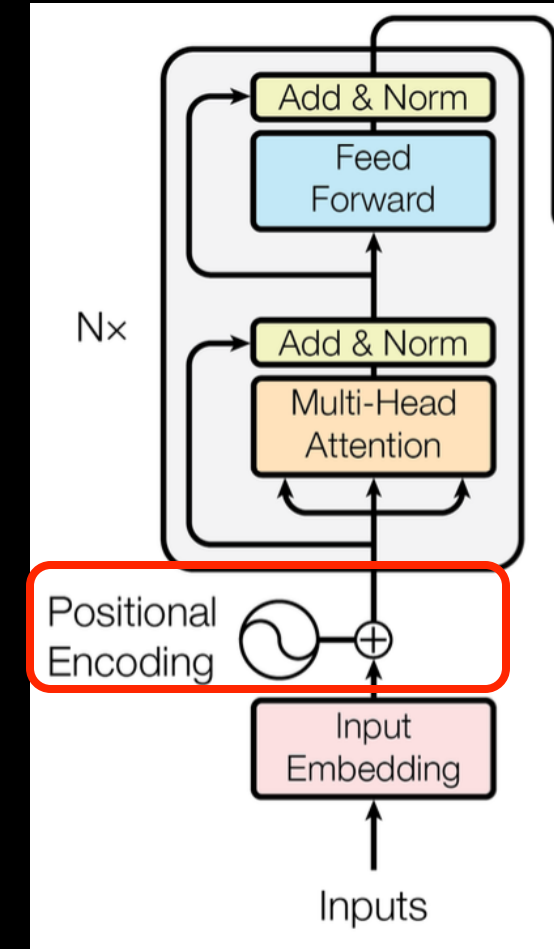
Concepts related to BPE

3. **Character:** It is a single letter, number or punctuation mark in text. We start by treating each word as a sequence of characters and merging process creates subword units based on character pairs.
4. **Frequency:** It refers to how many times a particular byte or character appears in the corpus. BPE is driven by the frequency of character pairs meaning that the most frequent pair of characters will be merged first.
5. **Merge:** It is the process of combining two consecutive characters or subword units into a new unit. Each merge reduces the total number of tokens in the corpus and leads to more abstract representations of words.

Transformers

➤ Positional Encoding

- **What it is:** Adds information about the position of each token in the sequence since transformers are position-agnostic.
- **Why:** Unlike RNNs, which inherently process sequences step-by-step, transformers process tokens in parallel and need positional context.
- **How:** Adds sinusoidal values (or learnable embeddings) to the input embeddings.
- **Example:**
 - Input embedding for "NLP": [0.5, 0.3, -0.8]
 - Positional encoding: [0.1, -0.2, 0.05]
 - Final encoding: [0.6, 0.1, -0.75].

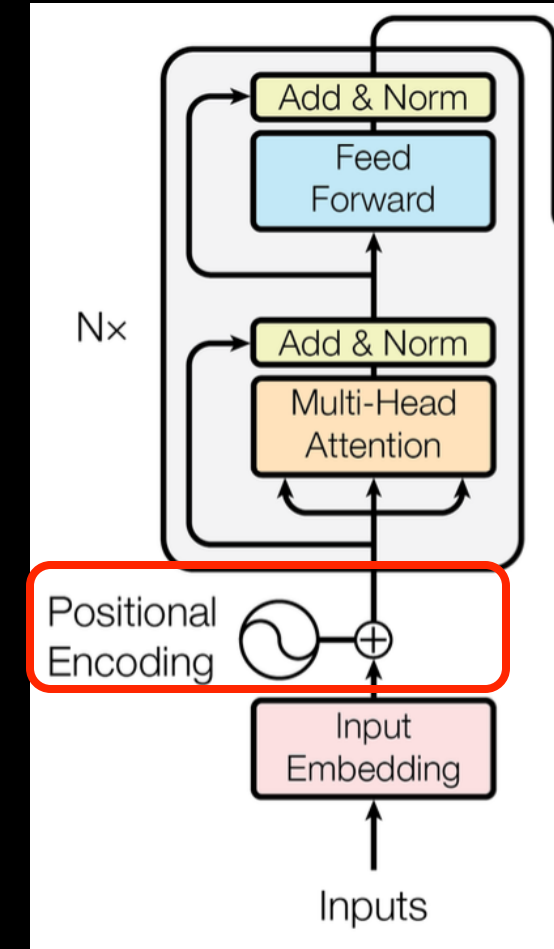


Transformers

➤ Positional Encoding

Transformers have no sense of order.
Solution:

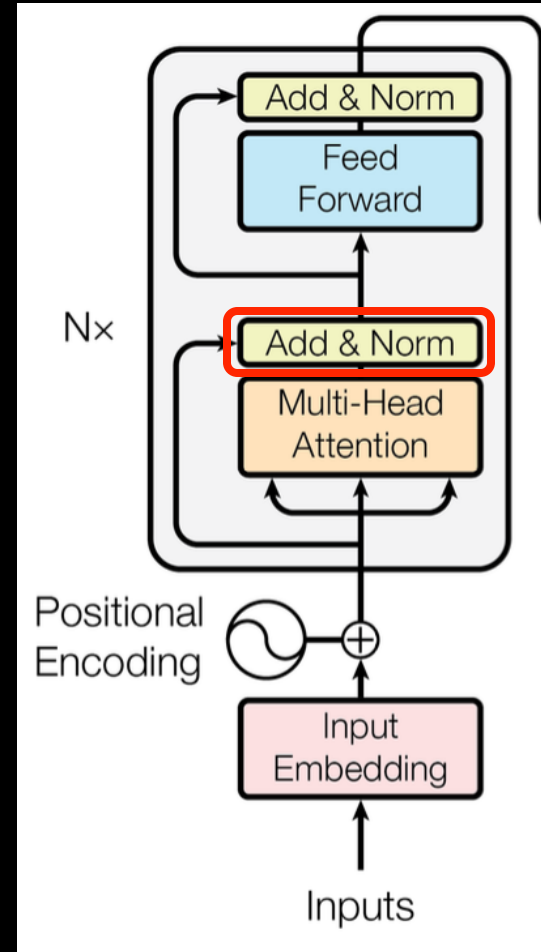
- Sinusoidal positional encoding
- Or learned positional embeddings



Transformers

➤ Add & Norm Layer

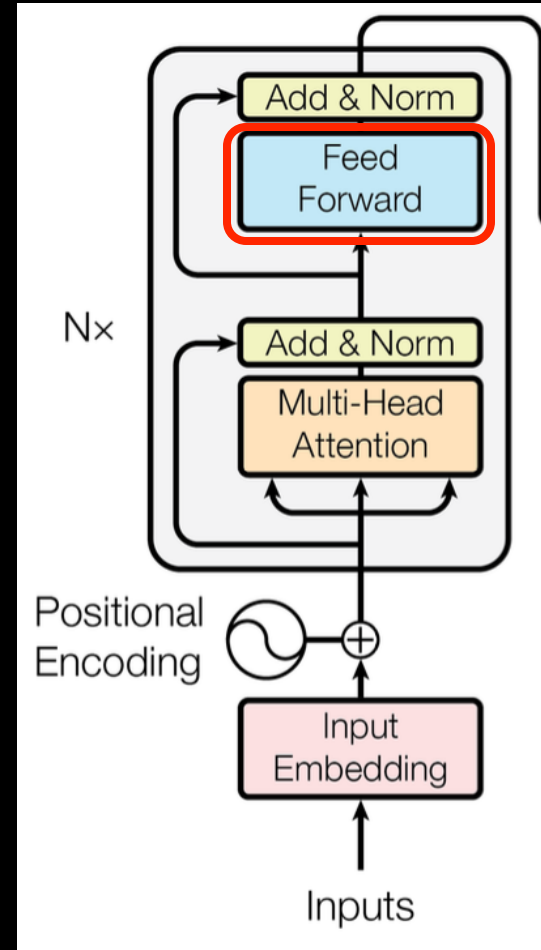
- **What it is:** Combines (adds) the original input with the output of the attention layer and normalizes the result.
- **Why:** Helps stabilize and accelerate training.



Transformers

➤ Feed Forward

- **What it is:** A fully connected neural network applied to each token's embedding independently.
- **Purpose:** Adds non-linearity and transforms the embeddings into a more expressive space.



Transformers

Attention Mechanism

Source: <https://arxiv.org/abs/1706.03762>

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
---	---	--	--

Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Łukasz Kaiser* Google Brain lukaszkaiser@google.com
--	---	--

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.





